Minimum Characters For Words

AlgoExpert · Medium · Strings

The problem

- Smallest multiset of characters that can spell each word (one at a time).

Intuition

- For each character, we need the maximum count it reaches in any single word.

Brute force vs optimal

- Concatenate everything → too many chars
- Per-char max frequency across words → O(total chars)

Approach

- 1. Count characters per word
- 2. Keep the max per character
- 3. Emit each character that many times

Worked example

char	max needed
h	1
i	1
s	1
t	1

```
after 'this'
```

After 'this'

Worked example

char	max needed
a	1
h	1
i	1
s	1
t	2

```
after 'that'
```

After 'that'

Worked example

char	max needed
a	1
d	2
e	2
h	1
i	1
s	1
t	2

```
after 'deed'
```

After 'deed'

Worked example

char	max needed
!	1
a	1
d	2
e	2
h	1
i	1
m	1
s	1
t	2

```
after 'them!'
```

After 'them!'

Java solution

```java
public class MinimumCharactersForWords {
    public static char[] minimumCharactersForWords(String[] words) {
        TreeMap<Character, Integer> need = new TreeMap<>();
        for (String w : words) {
            TreeMap<Character, Integer> f = new TreeMap<>();
            for (char c : w.toCharArray()) f.merge(c, 1, Integer::sum);
            f.forEach((c, n) -> need.merge(c, n, Math::max));
        }
        List<Character> out = new ArrayList<>();
        need.forEach((c, n) -> { for (int i = 0; i < n; i++) out.add(c); });
        char[] r = new char[out.size()];
        for (int i = 0; i < r.length; i++) r[i] = out.get(i);
        return r;
    }
}
```

Complexity

- Time: O(n * l)
- Space: O(c)

Edge cases & pitfalls

- Take max, not sum
- Special characters count
- Output order doesn't matter

Interview tips & follow-ups

- Follow-up: minimum chars to type words sequentially

Recap

- For each character, we need the maximum count it reaches in any single word.
- Complexity: O(n * l)